

eBay Interview — Question Map

Sr. Software Engineering Manager, Cloud Data Technology | Piyush Dubey | Thu Jun 18, 6PM IST

Your Story Bank (7 Stories)

Story	Primary Signal	One-line hook
DRIFT	Scope	Live map of cloud estate; 2B records/300 systems; real-time-vs-batch judgment
App Dependencies	Ambiguity	Blast-radius / criticality reframe (critical-path vs degraded-mode)
SCOUT	Ownership	POC-to-funded ML ops-intelligence; DS/product/eng operating model
TIBCO Migration	Technical Judgment	Zero-downtime migration; Strangler Fig; parallel slices
Pod 4 / Principal Eng	Conflict Resolution	Rogue parallel implementation; people vs architecture; coached + sequenced
Neo4j Failure	Growth / Failure	Wrong tech call; owned it publicly; pivoted to sustainable stack
Engineer Development	Mentorship	Task-executor -> owner; decision logs, pre-reads; reduced senior dependency

Note: TIBCO, Pod 4 & Mentorship all touch 'migration' — de-emphasize the migration framing on Pod 4 and Mentorship so they don't echo TIBCO.

Likely Questions -> Best Story Mapping

Likely Question	Lead Story	Backup
Most complex / impactful thing you've led	DRIFT	TIBCO
Breaking down a complex project	App Dependencies	TIBCO
Ambiguous problem with no clear definition	App Dependencies	DRIFT
Something you drove from scratch / initiative	SCOUT	DRIFT
A hard technical decision / trade-off	DRIFT (real-time vs batch)	TIBCO / Neo4j
High-risk migration / delivering under deadline	TIBCO	Pod 4
A conflict / disagreement you navigated	Pod 4	Greg
A cross-team / cross-org conflict	Pod 4	Security/CISO
A failure / wrong decision / what you'd do differently	Neo4j	SCOUT (early)
Developing / mentoring someone	Engineer Dev	—
Adapting your style to different individuals	Engineer Dev	—
Unifying data science, product & engineering	SCOUT	DRIFT
Improving reliability / reducing incidents	SCOUT	App Dependencies
Influencing without authority	Pod 4	DRIFT (adoption)
A time you changed your mind on evidence	Neo4j	Kafka/Solace
Handling a project going off-track	TIBCO	Neo4j

Known Gaps (have a plan if asked)

Signal / Question	Plan
-------------------	------

Perseverance: "a project that kept failing / you pushed through"	No dedicated story yet. Angle TIBCO (hard migration grind) or Neo4j recovery toward sustained
Pure team conflict (vs org-level)	You have many; pick one if he wants team-level rather than Pod 4's org-level.

Piyush Will Deep-Dive These (he’s technical — 13 yrs hands-on)

For each story, the pushback he’s likely to throw (red) and your anchor answer (green). Know these cold.

DRIFT

- Pushback: Why batch and not real-time everywhere? How do you do entity resolution across 300 systems? How was latency measured?
- ✓ Anchor: Deliberate risk-based split — real-time only for privileged access & de-provisioning; “the skill was knowing where it earned its keep.” Know the real measurement method for the latency claim.

App Dependencies

- Pushback: How did you actually determine criticality? Where did the data come from? Did you measure it or infer it?
- ✓ Anchor: App-owner attestation + observed connectivity + business-impact classification. Lead with the payment-gateway-vs-reporting example to make critical-path vs degraded-mode concrete.

SCOUT

- Pushback: Why classical ML and not LLMs/deep NLP? Does Lambda scale? How were the 55% / 30% measured?
- ✓ Anchor: Deliberate trade-off — cheap, fast, interpretable, fit the latency budget; architecture modular so model is swappable; LLMs are 'what's next', the leverage was the intelligence layer not the model. Keep the DS-ownership boundary clean ('I owned the platform, DS owned modelling').

TIBCO Migration

- Pushback: Why Solace over Kafka? What were the actual zero-downtime cutover mechanics? Explain the Strangler Fig.
- ✓ Anchor: Solace = sanctioned like-for-like JMS for request/reply. Walking skeleton end-to-end first; API façade + message bridge routed to both systems while migrating slice by slice.

Pod 4 / Principal Engineer

- Pushback: Was the principal actually right? Did you keep the worse design just to hit the deadline?
- ✓ Anchor: He WAS right — his approach was genuinely better. I shipped the agreed, working version to protect the timeline and sequenced his as a documented V2. He agreed: good-enough now, great later. Chose on merit + pragmatism, not process loyalty.

Neo4j Failure

- Pushback: Why reach for Neo4j at all? Why did Postgres end up working?
- ✓ Anchor: Honest ownership: over-indexed on the perfect technical fit, underweighted the team’s learning curve and operational readiness. ‘The right solution is the one the org can sustain, not just admire.’

Meet Piyush at His Familiarity Points

His World	Where Your Stories Connect
"Unifying data science, product, engineering"	SCOUT operating model — lead with this beat, use his exact framing
Reliability / 99.99% uptime / operational excellence	SCOUT (reactive→proactive), DRIFT, App Dependencies (blast radius)
AIML-powered systems	SCOUT (ML), DRIFT/Neo4j data work — speak as a fellow practitioner
Data lifecycle: ingestion → consumption (CDT)	DRIFT (300 sources → unified consumption), SCOUT pipeline
Rose through technical ranks	Go DEEP on architecture; he respects earned depth; be a collegial peer

Reminder: clean STAR/CARL, quantify results & know how each metric was measured, use 'I' not 'we', lead with the decision/insight not the architecture. Be his collegial future peer.